Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering
Course: Computer Architecture Lab (CO2008)

Lab 2
Branches and procedures

Student name: Tran Anh Tuan
Student ID: 2510261
Class / Group: L02
Instructor: Ton Huynh Long
Email: tuan.trank25cs@hcmut.edu.vn

Ho Chi Minh City, November 2026

Contents

1 Introduction **2**

2 Exercises **2**

2.1 Exercise 1 2

2.2 Exercise 2 3

2.3 Exercise 3 5

2.4 Exercise 4 6

1 Introduction

Important notes from the lab:

- Simulator: MARS MIPS (requires JDK).
- The register `$zero` can be written as `$0`.
- This week the programs use **branches** (`beq`, `bne`, `bgtz`, `bltz`, `ble`, `bgt`, ...), **jumps** (`j`) and **procedures** (`jal`/`jr`). A loop is a label plus a branch back to it.
- Calling convention used: arguments in `$a0`, `$a1`; result in `$v0`; return address in `$ra`. A procedure that calls itself saves `$ra` (and whatever it needs later) on the stack.
- Exercises 2 (GCD) and 4 (factorial) use **recursion**.
- Submit the answers to BKeL before the end of the lab section.

Source code: all assembly files (`EX2_1.asm` to `EX2_4.asm`) are stored at
https://github.com/tuan-bk25/Computer-architecture-lab-CO2008

2 Exercises

2.1 Exercise 1

> **Problem.** *Declare an integer array of 5 elements entered from the keyboard. If an element is not divisible by 4, change it to the closest number divisible by 4 (round down is prioritized), then print the result. For example, 35 becomes 36; 34 and 33 become 32.*

Idea. The remainder of x divided by 4 is just the last two bits of x, so `andi` with 3 gives it directly. Then there are only four cases:

$r = x \bmod 4$	Action	Example
0	keep	$32 \to 32$
1	$x - 1$ (down is closer)	$33 \to 32$
2	$x - 2$ (tie, round down)	$34 \to 32$
3	$x + 1$ (up is closer)	$35 \to 36$

Steps and syntax.

1. Reserve 5 words with `.space 20`. The program is split into three procedures, all called with `$a0` = base address and `$a1` = number of elements: `read_array`, `round_array`, `print_array`.
2. `read_array`: for each index print the prompt `array i :`, read an integer (`syscall 5`), store it with `sw`, then move the pointer by 4 bytes.

3. `round_array`: load each element, get the remainder, fix the value and store it back.

4. `print_array`: print every element followed by a space.

```
array: .space 20              # 5 words = 20 bytes
...
andi $t2, $t1, 3              # t2 = number % 4
beq  $t2, $0, store           # divisible by 4 -> keep
beq  $t2, $t5, up             # remainder 3 -> +1  ($t5 = 3)
sub  $t1, $t1, $t2            # remainder 1 or 2 -> round down
j    store
up:
addi $t1, $t1, 1
store:
sw   $t1, 0($a0)              # write back, then $a0 += 4
```

Note. `.space 20` is needed here. My first version used `.word 20`, which reserves only *one* word (holding the value 20), so storing 5 numbers overwrote the strings declared after the array. The `andi` trick also works for negative numbers because of two's complement (-5 has $r = 3$, so it becomes -4).

Result. Input $35, 34, 33, 32, 10$:

```
array 0 : 35
array 1 : 34
array 2 : 33
array 3 : 32
array 4 : 10
Result: 36 32 32 32 8
```

(10 has $r = 2$, a tie, so it goes down to 8.)

2.2 Exercise 2

> **Problem.** *Ask for two positive integers a and b (prompts `a =` and `b =` ; check that they are positive), then print their GCD and LCM using recursion. For example, $a = 6$, $b = 45$ gives $GCD = 3$, $LCM = 90$.*

Idea. Euclid's algorithm is naturally recursive:

$$\gcd(a, 0) = a, \qquad \gcd(a, b) = \gcd(b,\ a \bmod b)$$

The LCM comes from the GCD: $\operatorname{lcm}(a, b) = \dfrac{a}{\gcd(a, b)} \times b$. I divide first and multiply after, so the value stays small (no overflow from $a \times b$).

Steps and syntax.

1. `get_input` prints the prompt given in `$a0`, reads an integer and repeats while the value is not positive (`bgtz`). It is called once for a and once for b, so only the wrong number is asked again.

2. `gcd` takes a in `$a0` and b in `$a1`. Base case: $b = 0$ returns a. Otherwise it gets the remainder with `div`/`mfhi`, changes the arguments to (b, $a \bmod b$) and calls itself with `jal`.

3. `$ra` is saved on the stack before `jal`, because `jal` overwrites it and the procedure still needs it to return to its own caller. `$a0`/`$a1` do not need saving: nothing uses them after the call, the result just travels back in `$v0`.

4. In `main`: `gcd` result goes to `$s2`, then `div`/`mflo` gives $a/\gcd$ and `mult`/`mflo` gives the LCM.

```
gcd:
    bne  $a1, $0, gcd_rec     # b != 0 -> recurse
    move $v0, $a0             # b == 0 -> gcd = a
    jr   $ra
gcd_rec:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)          # save return address
    div  $a0, $a1
    move $a0, $a1             # a = b
    mfhi $a1                  # b = a % b
    jal  gcd                  # result is in $v0
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra
```

```
div  $s0, $s2                 # a / gcd
mflo $t0
mult $t0, $s1                 # (a / gcd) * b
mflo $s3                      # lcm
```

Calls for $a = 6$, $b = 45$ (the first call only swaps them):

call	$b = 0$?	next call
$\gcd(6, 45)$	no	$\gcd(45,\ 6)$
$\gcd(45, 6)$	no	$\gcd(6,\ 3)$
$\gcd(6, 3)$	no	$\gcd(3,\ 0)$
$\gcd(3, 0)$	yes	returns 3

Then LCM $= 6/3 \times 45 = 90$.

Result. Input -3 (invalid), then $a = 6$, $b = 45$:

```
a = -3
Invalid input! Please try again.
a = 6
b = 45
GCD = 3, LCM = 90
```

2.3 Exercise 3

> **Problem.** *Read 10 integers (prompt `Please insert a element:`), find the second largest element and all of its indexes. For example, the array* $1, 2, 7, -7, 3, 7, 4, 6, 8, 7$ *gives `Second largest value is 7, found in index 2, 5, 9`.*

Idea. "Second largest" here means the largest value that is *strictly smaller* than the maximum (in the example the max is 8 and the answer is 7, even though 7 appears three times). So after reading the array I use simple passes instead of one tricky pass:

1. Pass 1: find the maximum.
2. Pass 2: find the largest element that is smaller than the maximum.
3. Pass 3: print every index whose element equals that value.

Steps and syntax. Registers: `$s0` = 10 (size), `$s1` = array address, `$s2` = max, `$s3` = second largest, `$s4` = "second largest found" flag.

- Reading: element i is at base + $4i$, so `sll` by 2 gives the offset, `add` gives the address and `sw` stores the value.
- Pass 2 skips every element $\geq$ max. The first remaining element is taken directly (flag is 0); after that an element replaces `$s3` only if it is bigger.
- Printing indexes: a flag `$t3` says whether something was already printed, so a comma goes before every index except the first. This gives `2, 5, 9` with no trailing comma.

```
sec_loop:
    beq  $t0, $s0, check
    ...                         # t2 = array[t0]
    bge  $t2, $s2, sec_next     # skip if >= max
    beq  $s4, $0, sec_set       # first candidate
    ble  $t2, $s3, sec_next     # not better than current second
sec_set:
    move $s3, $t2
    li   $s4, 1                 # found
sec_next:
    addi $t0, $t0, 1
    j    sec_loop
```

```
bne  $t2, $s3, next_idx       # element != second largest
beq  $t3, $0, pw_idx          # first index -> no comma
la   $a0, comma               # print ", "
li   $v0, 4
syscall
pw_idx:
move $a0, $t0                 # print the index
li   $v0, 1
syscall
li   $t3, 1                   # something has been printed
```

Note. If all 10 numbers are equal there is no second largest value, so the program prints `No second largest value.` (the lab does not say what to do in that case).

Result. Array $1, 2, 7, -7, 3, 7, 4, 6, 8, 7$:

```
Second largest value is 7, found in index 2, 5, 9
```

Array $5, 1, 3, 3, -2, 5, 5, 1, 0, 2$ (max is 5, repeated):

```
Second largest value is 3, found in index 2, 3
```

2.4 Exercise 4

> **Problem.** *Read a non-negative integer and print its factorial. For example, input 5 gives 120. For a negative input like -2, print `A factorial for your number can not be found.`*

Idea. Factorial is recursive by definition:

$$0! = 1, \qquad n! = n \times (n-1)!$$

Each call must remember its own n and the address to return to, because after the inner call comes back it still has to multiply by n. Both are kept on the stack, so every call uses a small frame of 8 bytes.

Steps and syntax.

1. Read n (`syscall 5`). If $n < 0$ (`bltz`) print the error message and exit.
2. I also reject $n > 12$: $12! = 479{,}001{,}600$ fits in a 32-bit signed integer but $13! = 6{,}227{,}020{,}800$ does not, and `mult`/`mflo` would silently give a wrong answer.
3. `fact`: base case $n = 0$ returns 1 without touching the stack. Otherwise push `$ra` and n, call `fact`$(n-1)$, pop them, then multiply $n \times (n-1)!$.

```
fact:
    bgtz $a0, fact_rec
    li   $v0, 1               # 0! = 1
    jr   $ra
fact_rec:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $a0, 0($sp)          # save n
    addi $a0, $a0, -1
    jal  fact                 # v0 = (n-1)!
    lw   $a0, 0($sp)          # restore n
    lw   $ra, 4($sp)
    addi $sp, $sp, 8
    mult $a0, $v0
    mflo $v0                  # v0 = n * (n-1)!
    jr   $ra
```

How the calls go for $n = 3$ (down first, then back up):

going down	coming back
fact(3) saves $n = 3$, calls fact(2)	$3 \times 2 = 6$
fact(2) saves $n = 2$, calls fact(1)	$2 \times 1 = 2$
fact(1) saves $n = 1$, calls fact(0)	$1 \times 1 = 1$
fact(0) returns 1	

Note. A letter typed instead of a number makes `syscall 5` stop with an error in MARS, and plain MIPS cannot easily catch that, so the check only covers negative numbers (and too-large ones).

Result.

Input	Output
5	`120`
0	`1`
12	`479001600`
−2	`A factorial for your number can not be found.`
13	`Number is too big (max is 12).`